

VEGA: RunBook Following Agent

L. Rakasri, S. Ramya Ruba, R. Joha Shekin Saroj, S. Sivasuriya

Department of Computer Science and Engineering
VSB Engineering College, Karur

Abstract

Modern IT infrastructures require efficient and reliable execution of operational runbooks to ensure system stability and reduce human errors. This project presents an AI-powered Runbook Automation and Risk Analysis System that automates the execution and monitoring of operational runbooks. The system enables users to upload runbook documents, parse individual steps, and classify commands based on their risk levels before execution. By integrating the Ollama large language model, the system intelligently analyzes shell commands, database queries, REST API calls, and cloud operations to identify potential risks and provide safety recommendations. The platform supports automated execution of approved tasks while maintaining detailed execution logs and audit trails. A web-based interface allows users to monitor runbook progress, review execution results, and manage historical records. The system also generates post-execution audit reports that summarize completed actions, skipped tasks, detected errors, and recommended follow-up activities. By combining artificial intelligence with automation, the proposed solution improves operational efficiency, enhances security, minimizes manual intervention, and supports reliable infrastructure management in modern IT environments.

1 Introduction

In modern IT environments, organizations rely on operational runbooks to perform routine maintenance, incident resolution and infrastructure management tasks. These runbooks contain a sequence of instructions that administrators must execute accurately to ensure system reliability and availability. However, manual execution of runbooks is often time-consuming, error-prone, and difficult to monitor, especially in large-scale infrastructures.

The rapid growth of cloud computing, distributed systems, and enterprise applications has increased the complexity of operational workflows. As a result, organizations require intelligent solutions that can automate repetitive tasks while ensuring safety and compliance. Artificial Intelligence (AI) has emerged as a powerful technology for enhancing automation by providing intelligent decision-making and risk assessment capabilities.

This project proposes an AI-Powered Runbook Automation and Risk Analysis System that combines automation with AI-based command analysis. The system allows users to upload runbook documents, extract executable steps, and evaluate commands before execution. By leveraging the Ollama large language model, the platform identifies potential risks associated with shell commands, database operations, API requests, and cloud management actions.

2 System Architecture

2.1 Overview

The AI-Powered Runbook Automation and Risk Analysis System is designed to automate the execution of operational runbooks while ensuring safety through intelligent risk assessment. The system follows a structured workflow that begins with the upload of a runbook document by the user. The uploaded runbook is parsed to extract individual operational steps and commands.

After extraction, each command is analyzed using the Ollama large language model to determine its risk level and potential impact on the target system. Based on the analysis results, commands are classified into categories such as low, medium, or high risk. The system then provides recommendations and warnings to assist administrators in making informed decisions before execution.

Once the commands are approved, the execution engine performs the specified tasks and con-

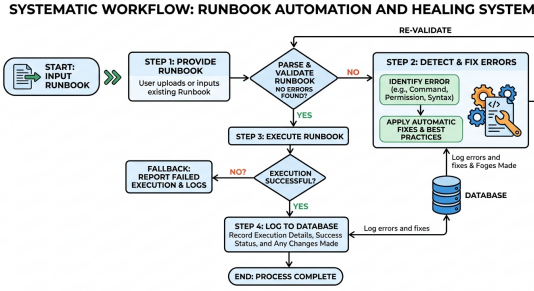


Figure 1: System Architecture Workflow: Runbook Parsing, Risk Analysis, and Execution Logging.

tinuously monitors their status. All execution details, including successful operations, failures, and skipped actions, are recorded in a centralized logging module. The collected information is further processed to generate audit reports that provide complete visibility into the execution process.

2.2 MCP-Model Context Protocol

The Model Context Protocol (MCP) module acts as a communication layer between the AI model and external tools, services, and data sources. It enables the Ollama-based language model to access relevant contextual information required for accurate runbook analysis and execution. Through MCP, the system can retrieve runbook content, execution history, configuration details, and system status information in a standardized manner.

The MCP module ensures seamless interaction between different components of the system by providing a structured mechanism for data exchange. It helps the AI model understand the operational context of commands before performing risk assessment. By supplying relevant contextual information, the module improves the accuracy of command classification and recommendation generation.

2.3 React FrontEnd

The React Frontend Module serves as the user interface of the AI-Powered Runbook Automation and Risk Analysis System. It is developed using React.js to provide a responsive, interactive, and user-friendly environment for managing runbooks and monitoring execution processes. The frontend acts as a bridge between users and the backend services, enabling seamless interaction with the system.

The module allows users to upload runbook documents, view extracted commands, and re-

view AI-generated risk assessments. It provides intuitive dashboards for displaying command classifications, execution status, system logs, and audit reports. Users can approve or reject commands based on the risk analysis results before execution.

2.4 Python BackEnd

The Python Backend Module serves as the core processing unit of the AI-Powered Runbook Automation and Risk Analysis System. It is responsible for handling business logic, processing user requests, managing workflows, and coordinating communication between the frontend, AI models, and external services. The backend is developed using Python due to its simplicity, scalability, and strong support for artificial intelligence and automation frameworks.

The module receives requests from the React frontend and processes uploaded runbook documents. It extracts runbook content, manages command execution workflows, and communicates with the Ollama model for risk analysis. The backend also validates user inputs, stores execution records, and ensures secure interaction between system components.

2.5 Ollama AI Agent

The Ollama AI Agent Module serves as the intelligence layer of the AI-Powered Runbook Automation and Risk Analysis System. It leverages locally deployed large language models through Ollama to analyze runbook commands, evaluate potential risks, and generate actionable recommendations. By processing command descriptions and execution contexts, the AI agent assists administrators in making informed decisions before executing operational tasks.

The module receives extracted commands from the Python backend and performs detailed analysis to determine their purpose, impact, and associated risk levels. Based on the evaluation, commands are classified into categories such as low, medium, or high risk. The AI agent also provides explanations and safety recommendations that help users understand the consequences of executing specific operations.

2.6 Error Detection

The module analyzes execution logs, command outputs, and system responses to identify potential issues in real time. When an error is detected, the system records detailed information, including the failed command, error message, execution timestamp, and possible causes.

In addition, the Error Detection Module works closely with the Ollama AI Agent to provide intelligent insights into detected failures. The AI agent can analyze error messages, suggest corrective actions, and recommend alternative approaches to complete the intended operation safely. This capability reduces troubleshooting time and improves operational efficiency.

The module also generates alerts and notifications to inform users about critical failures that require immediate attention. By providing proactive error monitoring and intelligent diagnostics, the Error Detection Module enhances system reliability, minimizes downtime, and supports successful runbook execution.

3 Data Flow

The data flow of the AI-Powered Runbook Automation and Risk Analysis System begins when a user uploads a runbook document through the React frontend interface. The uploaded document is transmitted to the Python backend, where the runbook parser extracts individual operational steps and executable commands.

The extracted commands are then forwarded to the Model Context Protocol (MCP) module, which gathers relevant contextual information such as system configurations, execution history, and operational metadata. This contextual data is combined with the extracted commands and sent to the Ollama AI Agent for analysis.

Finally, all execution logs, risk assessment results, detected errors, and command statuses are stored and processed to generate a comprehensive audit report. The report is then presented to the user through the frontend, providing complete visibility into the runbook execution process. This structured data flow ensures intelligent automation, accurate risk analysis, effective error handling, and reliable operational management.

4 Technology Stack

4.1 Frontend Technologies

The frontend of the AI-Powered Runbook Automation and Risk Analysis System is developed using React.js and Vite. React.js provides a component-based architecture for building interactive and responsive user interfaces, while Vite offers a fast development environment and optimized build process.

4.2 Backend Technologies

The backend is implemented using Python and Flask. Flask provides lightweight RESTful API services that facilitate communication between the frontend, database, AI agent, and external services. Flask-CORS is used to enable secure cross-origin communication between the client and server.

4.3 Database Technologies

SQLite is used as the primary database for storing runbooks, execution logs, audit records, risk assessment results, and user-related information.

4.4 Artificial Intelligence Technologies

The system utilizes Ollama as the local AI inference platform and Gemma 3:1B as the Large Language Model (LLM). The AI agent analyzes runbooks, identifies potential risks, detects execution errors, and generates remediation suggestions without requiring external cloud-based AI services.

SEQUENTIAL DATA FLOW: RUNBOOK AUTOMATION & SELF-HEALING

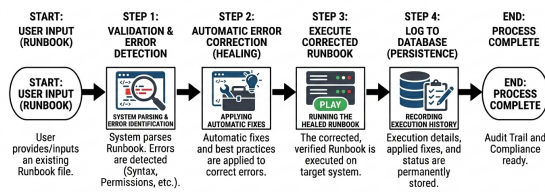


Figure 2: Sequential Data Flow Diagram: System Parsing, Automated Error Correction, Target Execution, and Database Persistence.

The Ollama AI Agent evaluates each command, determines its purpose, and assesses potential risks associated with execution. Based on the analysis, commands are classified into different risk levels, and recommendations are generated. The risk assessment results are returned to the Python backend and displayed to the user through the React frontend.